

RELATÓRIO DE TESTE TÉCNICO – DESENVOLVIMENTO

CANDIDATO: Guilherme Hinori

DATA: 09/03/2026

HORA INÍCIO: 19:00h

HORA ENTREGA: 22:30h

TEMPO MÁXIMO: 4 Horas

EXPERIÊNCIA PROFISSIONAL

Questão: Possui experiência em Desenvolvimento Web? Caso afirmativo, especifique a(s) linguagem(ns) e descreva sua experiência.

Resposta: Possuo experiência em **HTML, CSS e PHP**.

Questão: Possui experiência em Banco de Dados? Caso afirmativo, especifique o(s) banco(s) e descreva sua experiência.

Resposta: Possuo experiência em **MySQL**

1. CÁLCULO DE SALÁRIO LÍQUIDO

Enunciado: Faça um algoritmo que calcule o salário líquido de um trabalhador em função da quantidade de horas por dia, do preço da hora trabalhada e dos dias trabalhados no mês. Considerar um desconto de 3% sobre o salário bruto.

Linguagem utilizada: Python

Código Fonte:

Python

```
# Função que calcula o salário líquido
def calcular_salario_liquido(horas_por_dia, dias_trabalhados,
preco_hora):
    salario_bruto = horas_por_dia * dias_trabalhados * preco_hora
    desconto = salario_bruto * 0.03 # 3% de desconto
    salario_liquido = salario_bruto - desconto
    return salario_liquido
```

```
# Entrada de dados do usuário
```

```
horas = int(input("Digite o número de horas trabalhadas por dia: ")) [cite: 9]
dias = int(input("Digite o número de dias trabalhados no mês: ")) [cite: 9]
preco = float(input("Digite o preço da hora trabalhada: ")) [cite: 9]

# Processamento
salario = calcular_salario_liquido(horas, dias, preco) [cite: 10]

# Saída
print(f"O salário líquido do trabalhador é: R$ {salario:.2f}") [cite: 11]
```

2. VERIFICAÇÃO DE VALORES (A, B, C, D)

Enunciado: Receba 4 valores inteiros A, B, C e D. Se $B > C$, $D > A$, $(C+D) > (A+B)$, C e D forem positivos e A for par, exibir "Valores aceitos". Caso contrário, "Valores não aceitos".

Linguagem utilizada: Python

Código Fonte:

Python

```
# Entrada de dados
A = int(input("Digite o valor de A: ")) [cite: 14]
B = int(input("Digite o valor de B: ")) [cite: 15]
C = int(input("Digite o valor de C: ")) [cite: 16]
D = int(input("Digite o valor de D: ")) [cite: 17]

# Verificação de condições (Lógica segmentada para clareza) [cite: 18]
cond1 = B > C [cite: 19]
cond2 = D > A [cite: 20]
cond3 = (C + D) > (A + B) [cite: 21]
cond4 = C > 0 and D > 0 [cite: 22]
cond5 = A % 2 == 0 [cite: 23]

# Avaliação Final [cite: 24]
if cond1 and cond2 and cond3 and cond4 and cond5: [cite: 25]
    print("Valores aceitos") [cite: 26]
else: [cite: 27]
    print("Valores não aceitos") [cite: 28]
```

3. DECOMPOSIÇÃO DE NOTAS (CÉDULAS)

Enunciado: Faça um algoritmo que receba um valor inteiro. Dado o valor, calcule o menor número de notas (cédulas) possíveis no qual o valor pode ser decomposto. As notas consideradas são de 200, 100, 50, 20, 10, 5, 2 e 1. A seguir mostre o valor lido e a relação de notas necessárias.

Linguagem utilizada: Python

Código Fonte:

Python

```
# Entrada do valor a ser decomposto
valor_inicial = int(input("Digite um valor inteiro: "))
valor = valor_inicial

# Lista de cédulas disponíveis no sistema
notas = [200, 100, 50, 20, 10, 5, 2, 1]

print(f"Valor lido: R$ {valor_inicial}")
print("-" * 30)

# Algoritmo de decomposição (Greedy)
for nota in notas:
    quantidade = valor // nota # Divisão inteira para obter a quantia
    de notas
    print(f"{quantidade} nota(s) de R$ {nota},00")
    valor %= nota # O resto da divisão torna-se o novo valor a ser
    decomposto
```

4. CÁLCULO DA EXPRESSÃO (X)

Enunciado: Considerando a expressão $AX + BX + C = 1$, faça um algoritmo que receba 3 valores inteiros A, B, e C e calcule o valor de X. Caso A e B possuam valor 0 e C seja diferente de 1, imprimir “Solução impossível”.

Linguagem utilizada: Python

Lógica Matemática:

Para isolar X, seguimos os passos:

$$X(A + B) + C = 1$$

$$X(A + B) = 1 - C$$

$$X = \frac{1 - C}{A + B}$$

Código Fonte:

Python

```
# Entrada de dados
A = int(input("Digite o valor de A: "))
B = int(input("Digite o valor de B: "))
C = int(input("Digite o valor de C: "))

# Verificação de viabilidade da solução
# A solução é impossível se (A + B) for 0, pois não existe divisão por
zero
if (A + B) == 0:
    if C != 1:
        print("Solução impossível")
    else:
        # Se A+B=0 e C=1, qualquer valor de X satisfaz a equação (0 =
        0)
        print("Solução indeterminada (Infinitas possibilidades)")
else:
    # Cálculo de X
    X = (1 - C) / (A + B)
    print(f"O valor de X é: {X:.2f}")
```

5. CONVERSOR DE PADRÃO DE PLACAS (BRASIL <-> MERCOSUL)

Enunciado: Faça um algoritmo que receba uma placa e imprima o padrão informado e sua versão correspondente no outro padrão. A placa deve ser validada no formato AAA9999 (Brasil) ou AAA9A99 (Mercosul).

Linguagem utilizada: Python

Lógica de Conversão:

- **De Brasil para Mercosul:** O 5º caractere (índice 4) muda de **número** para **letra** seguindo o mapeamento (0=A, 1=B, ..., 9=J).
- **De Mercosul para Brasil:** O 5º caractere (índice 4) muda de **letra** para **número** (A=0, B=1, ..., J=9).

Código Fonte:

Python

```
def converter_placa():
    placa = input("Digite a placa do veículo: ").upper()

    # Mapeamento para conversão do 5º caractere
    map_num_para_letra = "ABCDEFGHIJ"

    # Validação básica de tamanho
    if len(placa) != 7:
        print("formato inválido")
        return

    # Verificação do prefixo (3 primeiras letras) e sufixo (2 últimos números)
    prefixo_ok = placa[:3].isalpha()
    quarto_char_ok = placa[3].isdigit()
    sufixo_ok = placa[5:7].isdigit()

    if not (prefixo_ok and quarto_char_ok and sufixo_ok):
        print("formato inválido")
        return

    char_conversao = placa[4]

    # Lógica para PADRÃO BRASIL (AAA9999)
    if char_conversao.isdigit():
        indice = int(char_conversao)
        corresp = placa[:4] + map_num_para_letra[indice] + placa[5:]
        print("Padrão: Brasil")
        print(f"Correspondente: {corresp}")

    # Lógica para PADRÃO MERCOSUL (AAA9A99)
    elif char_conversao.isalpha() and char_conversao in map_num_para_letra:
        indice = map_num_para_letra.find(char_conversao)
```

```
        corresp = placa[:4] + str(indice) + placa[5:]
        print("Padrão: Mercosul")
        print(f"Correspondente: {corresp}")

    else:
        print("formato inválido")

converter_placa()
```

6. ANÁLISE DE PROPRIEDADES NUMÉRICAS (1 A N)

Enunciado: Construa um algoritmo que receba um número inteiro positivo N e gere uma sequência de 1 a N. Para cada número, avalie se ele é um **número perfeito**, **múltiplo de 3**, **múltiplo de 5** ou se possui **raiz quadrada inteira**.

Linguagem utilizada: Python

Código Fonte:

Python

```
import math

# Entrada de dados
n_input = int(input("Digite um número inteiro positivo (N): "))

print(f"\nAnalisando a sequência de 1 até {n_input}:")
print("-" * 40)

for i in range(1, n_input + 1):
    propriedades = []

    # 1. Verificação de Número Perfeito
    soma_divisores = 0
    for j in range(1, i):
        if i % j == 0:
            soma_divisores += j
    if i > 1 and soma_divisores == i:
        propriedades.append("número perfeito")

    # 2. Verificação de Múltiplo de 3
    if i % 3 == 0:
        propriedades.append("multiplo de 3")

    # 3. Verificação de Múltiplo de 5
    if i % 5 == 0:
        propriedades.append("multiplo de 5")

    # 4. Verificação de Raiz Quadrada Inteira
    raiz = math.sqrt(i)
    if raiz == int(raiz):
        propriedades.append("raiz inteira")

    # Exibição dos resultados encontrados para o número i
    if propriedades:
        print(f"Número {i}: {' , '.join(propriedades)}")
```

7. FORMATAÇÃO DE TÍTULOS CENTRALIZADOS

Enunciado: Escreva um programa que receba um título dividido em duas partes (superior e inferior) e os imprima centralizados entre hifens. O número total de caracteres de cada linha deve ser o dobro do número de caracteres do próprio título, e ambas as linhas devem ter o mesmo comprimento final.

Linguagem utilizada: Python

Código Fonte:

Python

```
# Entrada de dados
titulo_sup = input("Digite o título superior: ").strip()
titulo_inf = input("Digite o título inferior: ").strip()

# De acordo com a regra: o total de caracteres deve ser o dobro do
# título.
# Para manter a simetria entre as duas linhas, baseamos o cálculo no
# maior título.
tamanho_base = max(len(titulo_sup), len(titulo_inf))
largura_total = tamanho_base * 2

def imprimir_centralizado(texto, largura_alvo):
    # Calcula quantos hifens sobram para as laterais
    espacos_laterais = largura_alvo - len(texto)

    # Divide por 2 para cada lado (ajusta se for ímpar para manter
    # simetria)
    lado_esquerdo = espacos_laterais // 2
    lado_direito = largura_alvo - len(texto) - lado_esquerdo

    print(f"{'-' * lado_esquerdo} {texto} {'-' * lado_direito}")

# Execução da formatação
print("\nResultado:")
imprimir_centralizado(titulo_sup, largura_total)
imprimir_centralizado(titulo_inf, largura_total)
```

8. MATRIZ QUADRADA EM FORMATO DE CARACOL

Enunciado: Elabore um algoritmo que construa uma matriz quadrada de tamanho N em formato de caracol (espiral). Uma matriz quadrada é aquela que possui o número de linhas igual ao número de colunas.

Linguagem utilizada: Python

Código Fonte:

Python

```
# Entrada do tamanho da matriz
N = int(input("Digite o tamanho da matriz quadrada (N): "))

# Inicialização da matriz N x N com zeros
matriz = [[0] * N for _ in range(N)]

num = 1
# O loop percorre as 'camadas' da matriz, de fora para dentro
for i in range((N + 1) // 2):

    # 1. Preenche a linha superior (da esquerda para a direita)
    for j in range(i, N - i):
        matriz[i][j] = num
        num += 1

    # 2. Preenche a coluna da direita (de cima para baixo)
    for j in range(i + 1, N - i):
        matriz[j][N - i - 1] = num
        num += 1

    # 3. Preenche a linha inferior (da direita para a esquerda)
    for j in range(N - i - 2, i - 1, -1):
        matriz[N - i - 1][j] = num
        num += 1

    # 4. Preenche a coluna da esquerda (de baixo para cima)
    for j in range(N - i - 2, i, -1):
        matriz[j][i] = num
        num += 1

# Exibição formatada da matriz
print(f"\nMatriz Caracol {N}x{N}:")
for linha in matriz:
    # Formatação com join para remover os colchetes e melhorar a
    # leitura
    print(" ".join(f"{item:02d}" for item in linha))
```

9. ANÁLISE DE CONSULTA SQL (DER)

Questão: Analise a instrução SQL fornecida e selecione a alternativa que descreve corretamente o retorno da consulta com base no Diagrama Entidade Relacionamento (DER).

Resposta Selecionada:

[X] Será retornado o código, o nome e o CEP dos clientes que moram no bairro PACARAIMA e, são da cidade PORTO DE MOZ do estado do Espírito Santos ou são da cidade URUARA do Distrito Federal.

10. ANÁLISE DE BANCO DE DATOS (DER)

Enunciado: Analise o Diagrama Entidade-Relacionamento (DER) que apresenta as tabelas **CDCLIENTE**, **LCATENDIMENTO** e **CDFUNCIONARIO**.

A) Relacionamento entre Tabelas

Questão: É certo dizer que o relacionamento entre a tabela **CDCLIENTE** para **LCATENDIMENTO**, é:

Resposta Selecionada: ☒ 1:N (Um para muitos)

Justificativa Técnica: Observando a notação do diagrama, um registro na tabela de Clientes pode estar associado a múltiplos registros na tabela de Atendimentos (indicado pelo símbolo de "pé de galinha" ou conectores múltiplos na extremidade de **LCATENDIMENTO**).

B) Afirmações sobre as Regras de Negócio

Questão: Assinale com X as afirmações corretas com base nas restrições de integridade (NN = NOT NULL) e chaves estrangeiras:

Respostas Selecionadas:

- ☒ **Todo atendimento requer que exista um cliente e um funcionário cadastrado previamente.**

Justificativa: Os campos **LCAFUNCIONARIO** e **LCACLIENTE** na tabela **LCATENDIMENTO** são Foreign Keys (FK) e possuem a restrição **NN (Not Null)**, o que obriga a existência de um vínculo válido com as tabelas primárias.

- ☒ **Um cliente pode ser atendido por vários funcionários, sendo um funcionário por vez.**

Justificativa: A tabela **LCATENDIMENTO** resolve um relacionamento Muitos-para-Muitos (M:N) entre clientes e funcionários. Cada registro de atendimento associa um cliente a um único funcionário naquela data/hora específica.

11. CONSULTAS SQL (DML)

Enunciado: Com base nos dados das tabelas **CDCLIENTE**, **CDFUNCIONARIO** e **LCATENDIMENTO**, escreva as instruções SQL abaixo:

A) Clientes sem atendimento

Instrução:

SQL

```
SELECT CDCLINOME  
FROM CDCLIENTE  
WHERE CDCLICODIGO NOT IN (SELECT LCACLIENTE FROM LCATENDIMENTO);
```

Ou utilizando JOIN:

SQL

```
SELECT C.CDCLINOME  
FROM CDCLIENTE C  
LEFT JOIN LCATENDIMENTO A ON C.CDCLICODIGO = A.LCACLIENTE  
WHERE A.LCACLIENTE IS NULL;
```

B) Produtividade de Funcionários

Instrução:

SQL

```
SELECT F.CDFNOME, COUNT(A.LCACODIGO) AS QUANTIDADE_ATENDIMENTOS  
FROM CDFUNCIONARIO F  
LEFT JOIN LCATENDIMENTO A ON F.CDFCODIGO = A.LCAFUNCIONARIO  
GROUP BY F.CDFNOME;
```

(O uso do LEFT JOIN garante que funcionários com zero atendimentos também apareçam na lista).

C) Filtro por Período (Bimestre e Semestre)

Instrução:

SQL

```
SELECT C.CDCLINOME, A.LCADATAHORAATEND  
FROM CDCLIENTE C  
INNER JOIN LCATENDIMENTO A ON C.CDCLICODIGO = A.LCACLIENTE  
WHERE (A.LCADATAHORAATEND BETWEEN '2011-01-01 00:00:00' AND '2011-02-28 23:59:59')  
OR (A.LCADATAHORAATEND BETWEEN '2011-07-01 00:00:00' AND '2011-12-31')
```

12. GESTÃO DE PROJETOS E PRAZOS

Cenário: * Prazo total prometido ao cliente: 10 dias.

- **Escopo:** 3 relatórios de complexidade similar.
- **Status atual:** 1º relatório finalizado em 6 dias.
- **Restrição:** O setor de testes necessita de 3 dias fixos para validação.

Análise da Situação: Com base no desempenho do primeiro relatório (6 dias), a estimativa real para os dois restantes é de 12 dias. Somando os 3 dias de teste, o tempo

total necessário seria de 15 dias úteis a partir do início, excedendo o prazo de 10 dias acordado com o cliente.

Ações Sugeridas para a Entrega:

1. **Comunicação Transparente e Escalação:** Informar imediatamente ao gestor de projetos ou PO (Product Owner) sobre o desvio no cronograma. É fundamental que a empresa saiba do risco de atraso o quanto antes para gerenciar a expectativa do cliente.
2. **Entrega Incremental (Pipeline de Testes):** Enviar o primeiro relatório já finalizado imediatamente para o setor de testes. Enquanto os testes do primeiro ocorrem (nos 3 dias previstos), trabalharei no desenvolvimento do segundo, otimizando o tempo morto.
3. **Negociação de Prioridades:** Verificar com o cliente qual dos relatórios restantes é mais crítico para a reunião de fechamento. Caso o prazo não possa ser estendido, propor a entrega completa dos dois mais importantes e um prazo adicional para o terceiro.
4. **Força-Tarefa ou Auxílio:** Solicitar o apoio de outro desenvolvedor para assumir um dos relatórios restantes, permitindo o desenvolvimento em paralelo para tentar enquadrar a entrega nos dias que restam.

13. GESTÃO DE PROJETOS E TOMADA DE DECISÃO

Cenário: * **Solicitação:** 3 relatórios similares.

- **Prazo da Empresa:** 10 dias.
- **Prazo do Cliente:** 15 dias.
- **Status Atual:** 1º relatório finalizado em 6 dias.
- **Necessidade de Testes:** 3 dias.

Análise da Situação: Se o primeiro relatório levou 6 dias e os próximos têm complexidade similar, a projeção para finalizar o desenvolvimento dos três é de **18 dias** (6 x 3). Somando os 3 dias necessários para o setor de testes, a entrega ocorreria em **21 dias**, o que excede tanto o prazo da empresa (10 dias) quanto a necessidade do cliente (15 dias).

Ações Tomadas:

1. **Comunicação de Risco (Escalação):** Notificar imediatamente o gestor ou o time de produto sobre a inviabilidade do prazo de 10 dias, apresentando a métrica real de desenvolvimento (6 dias/relatório).
2. **Entrega em Fluxo (Pipeline):** Disponibilizar o primeiro relatório já finalizado para o setor de testes imediatamente. Isso permite que a validação ocorra em paralelo ao desenvolvimento do segundo relatório, otimizando o cronograma.
3. **Negociação de Prioridade:** Sugerir ao time de atendimento que verifique com o cliente qual dos três relatórios é o mais crítico para a reunião. O foco seria

garantir a entrega perfeita de ao menos um ou dois relatórios dentro do prazo de 15 dias, em vez de entregar os três com atraso ou sem testes.

4. **Solicitação de Apoio:** Avaliar com a gestão a possibilidade de alocar outro desenvolvedor para assumir um dos relatórios restantes, visando reduzir o tempo total de execução através do trabalho em paralelo.

14. ANÁLISE DE IMPACTO E MANUTENÇÃO DE SOFTWARE

Cenário: Alteração de campos obrigatórios (categoria, descrição e grupo) para opcionais em uma rotina de cadastro. Após 40 dias, relatórios de estoque/produtos começaram a apresentar divergências de quantidades para alguns clientes.

A) Análise para auxiliar o suporte

Para diagnosticar o erro, eu realizaria os seguintes passos técnicos:

- **Cruzamento de Dados (SQL):** Executaria uma consulta no banco de dados para identificar se os produtos com erro nos relatórios são justamente aqueles cadastrados nos últimos 40 dias que possuem os campos "categoria" ou "grupo" nulos ou vazios.
- **Rastreamento de Log de Alterações:** Verificaria o histórico de entrada desses produtos para confirmar se a divergência coincide com o uso da nova funcionalidade de "cadastro ágil" (campos opcionais).
- **Revisão de Código dos Relatórios:** Analisaria as queries (consultas SQL) dos relatórios afetados. O objetivo é verificar se eles utilizam `INNER JOIN` com as tabelas de Categoria ou Grupo, ou se possuem filtros `GROUP BY` que ignoram registros com valores nulos.

B) Diagnóstico e Ações Sugeridas

Diagnóstico do Problema: O problema é um **Efeito Colateral de Integridade Referencial ou Lógica**. Anteriormente, os relatórios foram desenvolvidos sob a premissa de que todo produto possuía obrigatoriamente uma categoria e um grupo. Ao tornar esses campos opcionais na interface, foram criados registros no banco com valores `NULL`. Relatórios que utilizam `INNER JOIN` entre a tabela de produtos e categorias, por exemplo, ocultam automaticamente qualquer produto que não tenha uma categoria vinculada. Isso causa a "perda" de quantidades nas somatórias finais dos relatórios, gerando a divergência relatada.

Ações para Resolver:

1. **Ação Imediata (Correção de Dados):** Realizar um "Update" nos registros órfãos, atribuindo-os a uma categoria/grupo padrão (ex: "Geral" ou "Indefinido") para que voltem a aparecer nos relatórios.
2. **Ação Técnica (Ajuste de Consultas):** Alterar as instruções SQL dos relatórios de `INNER JOIN` para `LEFT JOIN`, garantindo que o produto seja contabilizado mesmo que não possua categoria ou grupo vinculado.
3. **Ação Preventiva (Refatoração):** Implementar o tratamento de valores nulos no código (usando funções como `COALESCE` ou `IFNULL`) e reavaliar se a regra de negócio realmente permite que esses campos sejam vazios sem comprometer a contabilidade e a organização do estoque do cliente.